Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 3
по дисциплине «Методы машинного обучения»
на тему
«Градиентный спуск»
Вариант 17

Выполнил: студент гр. 351002 Лащенко Е.Н.
Проверил: Лебедевич А.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью работы является реализация пакетного и стохастического градиентного спуска для линейной регрессии, исследование влияния скорости обучения на сходимость и применение L2-регуляризации. Индивидуальное задание варианта 17 выполнено на синтетической выборке из пяти признаков.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Линейная модель со свободным членом ставит объекту одно число. Прогноз записывают формулой

$$h(x) = w_0 + w_1 x_1 + \cdots + w_n x_n, \tag{2.1}$$

где h(x) – прогноз;

w_j – вес j-го признака;

x_j – значение признака;

n – число признаков.

Столбец x_0 заполняют единицами и не стандартизуют: иначе свободный член теряет смысл.

Минимизируют функцию потерь. Множитель 1/2 не сдвигает минимум и убирает двойку в производной. Штраф L2 добавляют ко всем весам, кроме свободного члена:

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m} (h(x_i) - y_i)^2 + \frac{\lambda}{2}\sum_{j=1}^{n} w_j^2, \tag{2.2}$$

где m – число объектов;

y_i – целевое значение i-го объекта;

λ – коэффициент L2-регуляризации.

В пунктах 2–5 штраф выключен: λ равен нулю. На графиках откладывают среднеквадратичную ошибку без множителя 1/2:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m} (h(x_i) - y_i)^2. \tag{2.3}$$

При нулевом λ частная производная равна

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m} (h(x_i) - y_i) x_{ij}. \tag{2.4}$$

В матричном виде тот же градиент равен (1/m) $X^T(Xw - y)$. Новые веса получают шагом против градиента:

$$w_{new} = w - \alpha \nabla J(w), \tag{2.5}$$

где α – скорость обучения.

Цикл завершают, если веса почти перестали меняться:

$$\|w_{new} - w\|_2 < \varepsilon\,. \tag{2.6}$$

Малый шаг сходится устойчиво и требует много итераций. Слишком большой шаг перескакивает минимум, и ошибка растёт.

Стохастический спуск на каждой итерации берёт один случайный объект и шагает по его градиенту:

$$w_{new} = w - \alpha(h(x_i) - y_i)x_i. \tag{2.7}$$

Среднее таких шагов совпадает с полным градиентом, но разброс оценки больше, поэтому устойчивый шаг меньше, чем у пакетного метода. Условие (2.6) для стохастического режима слабое: даже рядом с минимумом шаг по одному объекту не обращается в нуль. Сравнение методов поэтому ведут по фиксированному числу итераций, а в историю записывают ошибку (2.3) на всей выборке.

Для нечётного варианта к производной по весу признака добавляют λw_j:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m} \quad (h(x_i) - y_i)x_{ij} + \lambda\, w_j. \tag{2.8}$$

Свободный член по-прежнему не штрафуют. Рост λ стягивает веса признаков к нулю. При очень большом λ модель перестаёт использовать признаки и даёт константный прогноз.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Задание выполнено по варианту 17. Набор сгенерирован функцией make_regression: линейная зависимость и уровень шума известны, поэтому по кривой ошибки видно поведение оптимизатора. Вариант нечётный, поэтому регуляризация – L2.

1 Сгенерировать набор данных. Признаков должно быть не менее двух, без учёта целевого, то есть весов не менее трёх. При необходимости выполнить масштабирование.

2 Реализовать функцию пакетного градиентного спуска: задать начальные веса и порог ε, открыть цикл по итерациям, пересчитать веса по формуле (2.5) и завершить цикл при выполнении условия (2.6).

3 Исследовать несколько значений скорости обучения и сделать выводы.

4 Добавить выбор случайного объекта на каждой итерации и получить стохастический градиентный спуск по формуле (2.7).

5 Обучить линейную модель пакетным и стохастическим спуском, нанести среднеквадратичную ошибку обоих методов на один график и сравнить скорость сходимости.

6 Взять 20 коэффициентов L2-регуляризации в диапазоне от 0,001 до 1000 и показать, как веса зависят от коэффициента.

4 РЕШЕНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Решение записано на языке Python с библиотеками NumPy, pandas, matplotlib и scikit-learn. Фрагменты ниже продолжают один сценарий: имена из предыдущего фрагмента сохраняются.

4.1 Подготовка данных

Берут синтетическую регрессию make_regression. Признаков пять, вместе со свободным членом весов шесть. Параметр random_state равен 17 и фиксирует вариант. Шум генератора имеет стандартное отклонение 15.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 17
N_SAMPLES = 300
N_FEATURES = 5

X, y, coef_true = make_regression(
    n_samples=N_SAMPLES,
    n_features=N_FEATURES,
    n_informative=N_FEATURES,
    noise=15.0,
    random_state=RANDOM_STATE,
    coef=True,
)

feature_names = [f"x{j}" for j in range(1, N_FEATURES + 1)]
raw = pd.DataFrame(X, columns=feature_names)
raw["y"] = y

print(f"Размерность: {raw.shape[0]} объектов, {N_FEATURES}
признаков + целевая y")
print(raw.std(ddof=0).round(3).to_string())

scaler_x = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_x.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.reshape(-1, 1)).ravel()
```

В выборке 300 объектов. Стандартные отклонения признаков до масштабирования лежат около единицы, от 0,982 до 1,055. У целевой переменной отклонение равно 113,862: коэффициенты генерации составляют 28,608;

57,745; 25,620; 16,294 и 77,397. Один и тот же шаг α для такой шкалы у слишком крупный. Признаки и целевую переменную приводят к нулевому среднему и единичному стандартному отклонению. После стандартизации среднее матрицы признаков и среднее у равны нулю, стандартные отклонения равны единице.

4.2 Пакетный градиентный спуск

Функция считает градиент по всей выборке, пока параметр stochastic равен False. Начальные веса нулевые. Останов – по условию (2.6) с порогом ε = 10^{-8}. Полученные веса сверяют с LinearRegression и с аналитическим решением наименьших квадратов.

```
def gradient_descent(
    X, y, lr=0.1, n_iter=2000, eps=1e-6, reg=0.0,
    stochastic=False, random_state=17,
):
    # Пакетный или стохастический спуск. reg — коэффициент L2.
    # Свободный член не штрафуется.
    Xb = np.column_stack([np.ones(len(y)), np.asarray(X,
dtype=float)])
    y = np.asarray(y, dtype=float)
    m, n_weights = Xb.shape
    w = np.zeros(n_weights)
    history = [float(np.mean((Xb @ w - y) ** 2))]
    rng = np.random.default_rng(random_state)

    for iteration in range(1, n_iter + 1):
        if stochastic:
            idx = int(rng.integers(0, m))
            error_i = float(Xb[idx] @ w - y[idx])
            grad = error_i * Xb[idx]
        else:
            error = Xb @ w - y
            grad = (Xb.T @ error) / m
        grad[1:] += reg * w[1:]
        w_new = w - lr * grad
        if not np.all(np.isfinite(w_new)):
            history.append(np.inf)
            return w, np.asarray(history, dtype=float),
iteration
        delta = np.linalg.norm(w_new - w)
        w = w_new
        history.append(float(np.mean((Xb @ w - y) ** 2)))
        if eps > 0 and delta < eps:
            break
    return w, np.asarray(history, dtype=float), iteration
```

```
Xb = np.column_stack([np.ones(len(y_scaled)), X_scaled])
w_ls, *_ = np.linalg.lstsq(Xb, y_scaled, rcond=None)
mse_ls = float(np.mean((Xb @ w_ls - y_scaled) ** 2))

w_gd, hist_gd, iters_gd = gradient_descent(
    X_scaled, y_scaled, lr=0.1, n_iter=5000, eps=1e-8, reg=0.0,
stochastic=False
)

linreg = LinearRegression().fit(X_scaled, y_scaled)
w_sklearn = np.r_[linreg.intercept_, linreg.coef_]
```

Спуск останавливается на 151-й итерации. Конечная среднеквадратичная ошибка равна 0,019140 и совпадает с ошибкой аналитического МНК. Наибольшее расхождение весов с LinearRegression равно $5{,}647 \cdot 10^{-8}$. Свободный член равен нулю: и признаки, и целевая переменная центрированы. Веса приведены в таблице 4.1. Доля дисперсии y, которую модель не объясняет, равна 0,019140, то есть объяснённая доля составляет 0,981.

Таблица 4.1 – Веса линейной модели на стандартизованных данных

Вес	Градиентный спуск	LinearRegression	МНК
w_0	0,000000	0,000000	0,000000
w_1	0,236243	0,236243	0,236243
w_2	0,537819	0,537819	0,537819
w_3	0,220121	0,220121	0,220121
w_4	0,138492	0,138492	0,138492
w_5	0,720311	0,720311	0,720311

4.3 Скорость обучения

Тот же пакетный спуск запускают из нулевых весов при нескольких значениях α. Порог ε равен 10^{-8}, число итераций ограничено 2000. Итерацией выхода на минимум считают первый шаг, на котором ошибка превышает минимум МНК не более чем на 1 %. Результаты сведены в таблице 4.2.

```
learning_rates = [0.01, 0.1, 0.5, 1.0, 1.5]
histories = {}
for lr in learning_rates:
    w_lr, hist_lr, n_done = gradient_descent(
```

```
        X_scaled, y_scaled, lr=lr, n_iter=2000, eps=1e-8,
stochastic=False
    )
    histories[lr] = hist_lr

_, hist_div, _ = gradient_descent(
    X_scaled, y_scaled, lr=2.0, n_iter=12, eps=0.0,
stochastic=False
)
```

Таблица 4.2 – Пакетный спуск при разной скорости обучения

Скорость обучения	Число итераций	Итерация выхода	Ошибка на 200-й итерации	Конечная ошибка
0,01	1332	405	0,033412	0,019140
0,10	151	39	0,019140	0,019140
0,50	27	6	0,019140	0,019140
1,00	10	2	0,019140	0,019140
1,50	50	9	0,019140	0,019140

При $\alpha = 0{,}01$ к 200-й итерации ошибка ещё равна 0,033412 и заметно выше минимума МНК. Полный останов требует 1332 итерации. При $\alpha = 0{,}1$ минимум достигается за несколько десятков итераций. При $\alpha = 0{,}5$ и $\alpha = 1$ сходимость ещё быстрее: после стандартизации кривизна $J(w)$ порядка единицы, и шаг около единицы удобен для этого квадратичного функционала. При $\alpha = 1{,}5$ метод доходит до минимума, но останов наступает позже, чем при $\alpha = 1$: 50 итераций против 10. При $\alpha = 2$ ошибка растёт на каждом шаге, с 1 до 23,378 за 12 шагов: процесс расходится.

Кривые пяти сходящихся шагов на первых 200 итерациях показаны на рисунке 4.1. Шаг $\alpha = 0{,}01$ убывает плавно и к правой границе графика ещё не доходит до штриховой линии минимума МНК. Шаг $\alpha = 0{,}1$ выходит на эту линию примерно к 40-й итерации. Шаги 0,5, 1 и 1,5 почти сразу совпадают с минимумом, поэтому на рисунке они видны только у левой границы. Расходящийся шаг $\alpha = 2$ на график не нанесён: его ошибка выходит за шкалу от нуля до единицы.

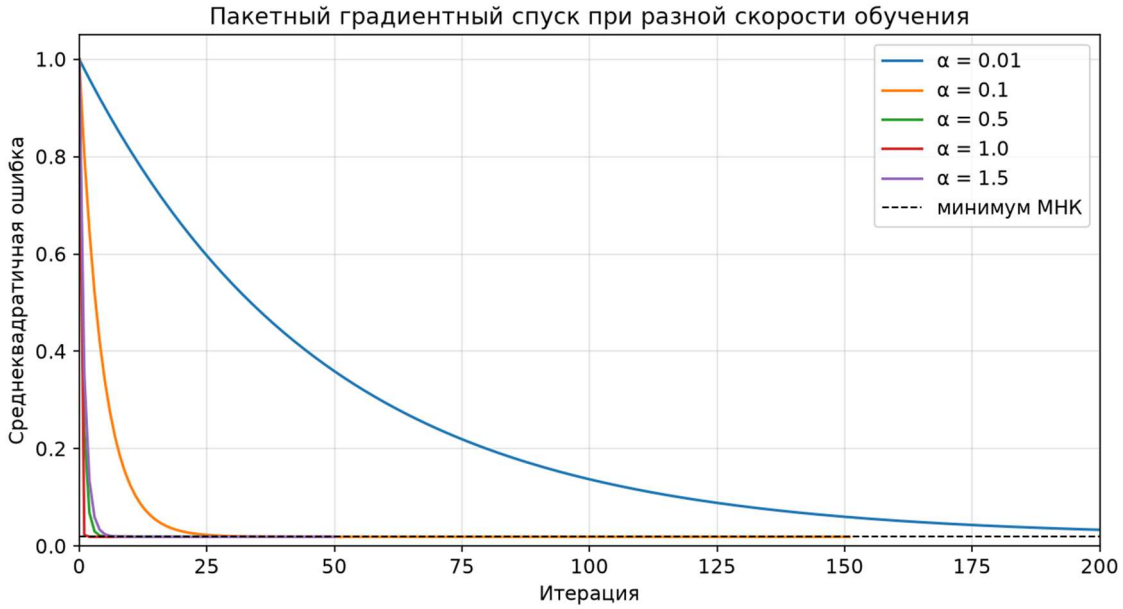


Рисунок 4.1 – Пакетный градиентный спуск при разной скорости обучения

Дальше для пакетного спуска берут α = 0,1: запас до границы расходимости остаётся, а минимум достигается быстро.

4.4 Стохастический градиентный спуск

В ту же функцию добавлен выбор одного случайного объекта. Индекс берут равновероятно, шаг идёт по формуле (2.7). Подходящее α подбирают по четырём запускам длиной 1500 итераций. Порог ε отключают: иначе сравнение оборвётся на случайном шаге. В историю по-прежнему пишут ошибку на всех 300 объектах.

```
sgd_rates = [0.001, 0.01, 0.05, 0.1]
for lr in sgd_rates:
    _, hist_try, _ = gradient_descent(
        X_scaled, y_scaled, lr=lr, n_iter=1500, eps=0.0,
        stochastic=True, random_state=RANDOM_STATE,
    )
```

Сводка запусков приведена в таблице 4.3. Среднюю ошибку хвоста считают по последним 100 итерациям. Ориентир – ошибка МНК, равная 0,019140.

Таблица 4.3 – Подбор скорости обучения стохастического спуска

Скорость обучения	Конечная ошибка	Средняя ошибка хвоста	Минимальная ошибка

Скорость обучения	Конечная ошибка	Средняя ошибка хвоста	Минимальная ошибка
0,001	0,068467	0,075110	0,068467
0,010	0,019496	0,019492	0,019160
0,050	0,022775	0,021100	0,019201
0,100	0,033591	0,025483	0,019583

При $\alpha = 0{,}001$ за 1500 итераций ошибка снижается только до 0,068467 и до минимума МНК не доходит. При $\alpha = 0{,}01$ средняя ошибка хвоста равна 0,019492 и лежит рядом с МНК. При $\alpha = 0{,}05$ и $\alpha = 0{,}1$ хвост остаётся выше: около 0,021 и 0,025. Для сравнения методов берут $\alpha = 0{,}01$.

4.5 Сравнение пакетного и стохастического спуска

Обе модели стартуют из нулевых весов. Пакетный спуск идёт с $\alpha = 0{,}1$ и порогом $\varepsilon = 10^{-8}$. Стохастический идёт с $\alpha = 0{,}01$, порог отключён, длина запуска – 1000 итераций.

```
N_COMPARE = 1000
w_batch, hist_batch, iters_batch = gradient_descent(
    X_scaled, y_scaled, lr=0.1, n_iter=N_COMPARE, eps=1e-8,
stochastic=False
)
w_stoch, hist_stoch, iters_stoch = gradient_descent(
    X_scaled, y_scaled, lr=0.01, n_iter=N_COMPARE, eps=0.0,
    stochastic=True, random_state=RANDOM_STATE,
)
```

Веса приведены в таблице 4.4. Пакетный спуск останавливается на 151-й итерации с ошибкой 0,019140. Стохастический проходит все 1000 итераций и заканчивает с ошибкой 0,019720. Порог «ошибка не более чем на 5 % выше МНК», то есть 0,020097, пакетный метод проходит на 32-й итерации, стохастический – на 347-й.

Таблица 4.4 – Веса пакетного и стохастического спуска

Вес	Пакетный спуск	Стохастический спуск	LinearRegression
w_0	0,00000	0,00388	0,00000
w_1	0,23624	0,24830	0,23624
w_2	0,53782	0,54642	0,53782
w_3	0,22012	0,21820	0,22012

Вес	Пакетный спуск	Стохастический спуск	LinearRegression
w_4	0,13849	0,12629	0,13849
w_5	0,72031	0,73392	0,72031

Один шаг пакетного метода читает все 300 объектов, один шаг стохастического – один объект. До указанного порога это 9600 просмотров строк против 347, то есть около 28 раз больше. Кривые ошибки показаны на рисунке 4.2. После останова пакетного спуска его линия продолжена постоянным значением: веса больше не меняются. Для стохастической кривой на верхнем графике проведено скользящее среднее по 30 итерациям.

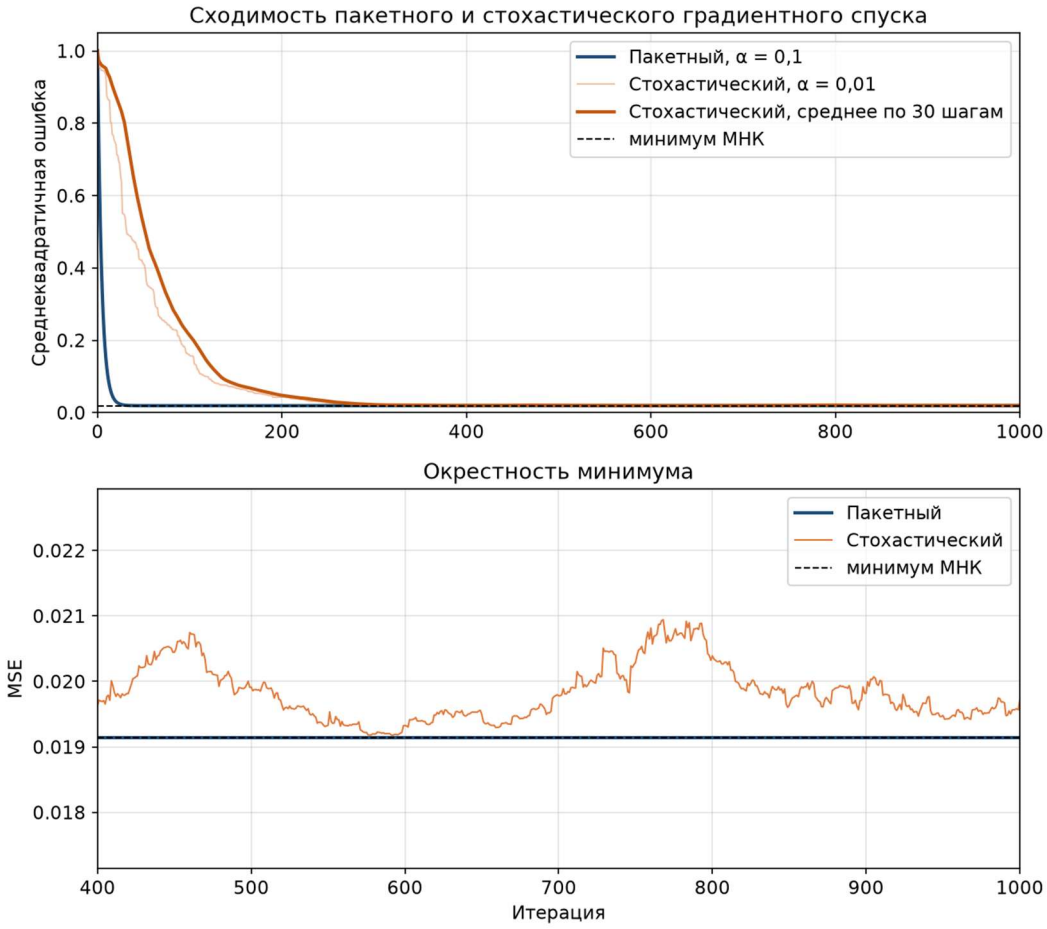


Рисунок 4.2 – Сходимость пакетного и стохастического градиентного спуска

Пакетный спуск даёт гладкую кривую и останавливается в минимуме. Стохастический доходит до той же окрестности позже по числу итераций. На нижнем графике рисунка 4.2 видно, что после 400-й итерации его ошибка не замирает, а колеблется около 0,019–0,021. Веса близки к LinearRegression, но с ним не совпадают. По числу итераций быстрее пакетный метод. По объёму

прочитанных данных быстрее стохастический. На этой выборке оба метода выходят на ошибку около 0,019.

4.6 L2-регуляризация

К функции потерь добавляют штраф по формуле (2.8). Берут 20 коэффициентов на логарифмической сетке от 0,001 до 1000. Для каждого λ веса считают пакетным спуском. Шаг уменьшают вместе с ростом λ, иначе член λw делает обновление расходящимся: берут минимум из 0,3 и величины 0,5/(1 + λ). Порог ε равен 10^{-10}.

Стационарная точка этого градиента должна совпасть с Ridge при параметре alpha, равном произведению числа объектов на λ, то есть 300λ.

```
lambdas = np.logspace(-3, 3, 20)
path = []
for lam in lambdas:
    lr_lam = min(0.3, 0.5 / (1.0 + lam))
    w_lam, hist_lam, n_lam = gradient_descent(
        X_scaled, y_scaled, lr=lr_lam, n_iter=5000,
        eps=1e-10, reg=float(lam), stochastic=False,
    )
    path.append(w_lam)

for lam in (0.001, 1.0, 100.0, 1000.0):
    lr_lam = min(0.3, 0.5 / (1.0 + lam))
    w_lam, _, _ = gradient_descent(
        X_scaled, y_scaled, lr=lr_lam, n_iter=5000,
        eps=1e-10, reg=float(lam), stochastic=False,
    )
    ridge = Ridge(alpha=N_SAMPLES * lam,
fit_intercept=True).fit(X_scaled, y_scaled)
    w_ridge = np.r_[ridge.intercept_, ridge.coef_]
```

Сверка для четырёх коэффициентов приведена в таблице 4.5. Наибольшее отклонение весов имеет порядок 10^{-10}, то есть стационарные точки совпадают.

Таблица 4.5 – Сверка градиентного спуска с Ridge

Коэффициент λ	Максимальное отклонение весов
0,001	$1{,}306 \cdot 10^{-10}$
1,000	$5{,}305 \cdot 10^{-11}$
100,000	$5{,}497 \cdot 10^{-11}$
1000,000	$4{,}494 \cdot 10^{-11}$

Веса признаков и ошибка на обучающей выборке для всей сетки приведены в таблице 4.6. Коэффициент λ в этой таблице округлён до трёх знаков после запятой. Свободный член в таблицу не входит: на всей сетке он остаётся около нуля, поскольку данные центрированы и w_0 в штраф не входит. Число итераций до останова убывает от 63 при λ = 0,001 до 24 при λ = 1000.

Таблица 4.6 – Веса признаков и ошибка при разных коэффициентах L2

λ	w_1	w_2	w_3	w_4	w_5	Ошибка
0,001	0,23600	0,53732	0,21993	0,13835	0,71962	0,01914
0,002	0,23575	0,53678	0,21972	0,13819	0,71889	0,01914
0,004	0,23522	0,53568	0,21929	0,13787	0,71738	0,01916
0,009	0,23413	0,53341	0,21841	0,13721	0,71427	0,01921
0,018	0,23192	0,52877	0,21661	0,13587	0,70792	0,01943
0,038	0,22747	0,51942	0,21298	0,13317	0,69513	0,02033
0,078	0,21878	0,50109	0,20583	0,12791	0,67010	0,02386
0,162	0,20277	0,46697	0,19247	0,11824	0,62363	0,03666
0,336	0,17611	0,40929	0,16967	0,10223	0,54540	0,07655
0,695	0,13846	0,32593	0,13626	0,07984	0,43304	0,17424
1,438	0,09602	0,22926	0,09679	0,05490	0,30366	0,34603
2,976	0,05877	0,14205	0,06051	0,03333	0,18767	0,55420
6,158	0,03260	0,07948	0,03408	0,01838	0,10483	0,73435
12,743	0,01697	0,04158	0,01790	0,00953	0,05479	0,85590
26,367	0,00852	0,02093	0,00903	0,00477	0,02756	0,92606
54,556	0,00420	0,01032	0,00446	0,00235	0,01359	0,96318
112,884	0,00205	0,00504	0,00218	0,00115	0,00663	0,98194
233,572	0,00099	0,00245	0,00106	0,00056	0,00322	0,99121
483,293	0,00048	0,00119	0,00051	0,00027	0,00156	0,99574
1000,000	0,00023	0,00057	0,00025	0,00013	0,00075	0,99794

Те же зависимости показаны на рисунке 4.3. Верхний график – веса признаков, нижний – ошибка на обучающей выборке. Ось λ логарифмическая. Крупные веса w_2 и w_5 убывают раньше остальных. Ошибка остаётся близкой к минимуму МНК, пока λ меньше единицы, и затем быстро растёт.

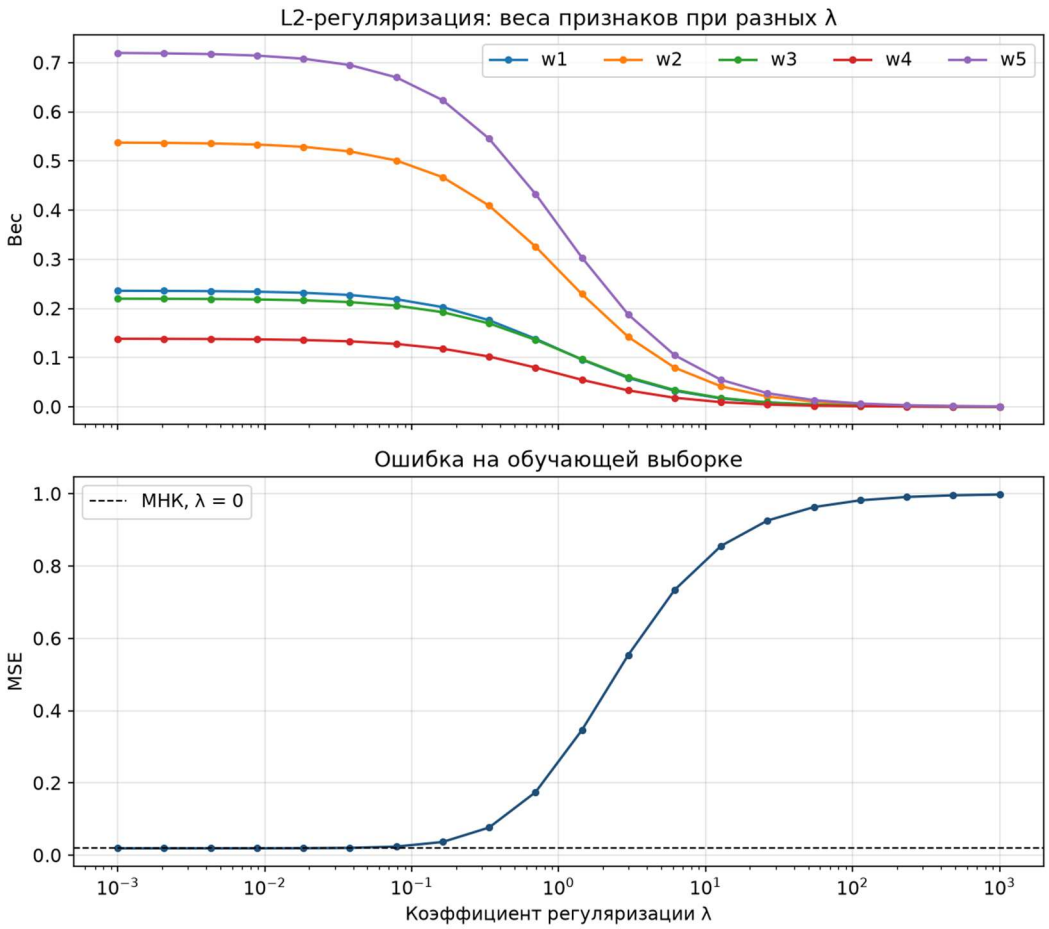


Рисунок 4.3 – Веса и ошибка при L2-регуляризации

На сетке от 0,001 до 1000 веса признаков монотонно стягиваются к нулю. При λ = 0,001 они практически совпадают с МНК, ошибка остаётся около 0,019. К λ = 1000 все штрафные веса по модулю меньше 0,001, а ошибка поднимается до 0,998 – почти до ошибки константного прогноза на стандартизованной целевой переменной, равной единице. Малый λ почти не отличается от обычного МНК. Промежуточные значения, примерно от 0,04 до 0,7, уже уменьшают норму весов, а рост ошибки ещё умеренный: с 0,020 до 0,174. Слишком большой λ упрощает модель до константы и перестаёт объяснять y.

5 ВЫВОДЫ

По варианту 17 на выборке из 300 объектов и пяти признаков реализован пакетный градиентный спуск. Из нулевых весов при $\alpha = 0{,}1$ он приходит к тому же решению, что LinearRegression и аналитический МНК: ошибка равна 0,019140. Масштабирование выполнено из-за большого разброса целевой переменной.

Скорость обучения задаёт режим. При $\alpha = 0{,}01$ сходимость медленная, при α от 0,1 до 1 – быстрая и устойчивая. При $\alpha = 2$ ошибка растёт на каждом шаге.

Стохастический спуск получен выбором одного случайного объекта. Его кривая ошибки шумная. По числу итераций он отстаёт от пакетного метода, зато каждый шаг читает один объект, а не все 300. До порога в 5 % над минимумом МНК это 347 объектов против 9600. Оба метода выходят на ошибку около 0,019. Пакетный останавливается в минимуме, стохастический колеблется вокруг него.

Для нечётного варианта включена L2-регуляризация. На 20 коэффициентах от 0,001 до 1000 веса признаков убывают к нулю, а ошибка на обучающей выборке растёт от уровня МНК до ошибки константного прогноза. Свободный член не штрафуется и остаётся около нуля. Сверка с Ridge подтверждает стационарную точку градиента. Поставленная цель достигнута.